

MLOps Foundations

1 Introduction

Machine learning has shifted from a research-centric activity to a core component of production systems such as recommendation engines, fraud detection pipelines, medical decision support tools, and autonomous systems. Despite impressive advances in model accuracy and scale, deploying ML systems reliably remains a major challenge. Industrial studies consistently report that a significant portion of engineering effort occurs after model development, particularly during deployment, monitoring, and maintenance [1].

Traditional software engineering assumes that system behavior is explicitly encoded in deterministic logic. In contrast, ML systems learn behavior from data, meaning that changes in data distributions can alter system behavior even when the code remains unchanged. These fundamental differences motivate the need for MLOps: a discipline focused on operationalizing ML systems throughout their lifecycle.

2 Background: Machine Learning and LLM Concepts for MLOps

This section briefly introduces the minimum machine learning (ML) and large language model (LLM) concepts required to understand MLOps. The goal is not to explain how models are built, but to clarify what artifacts exist, how behavior is learned, and why operational challenges arise.

2.1 What Is Machine Learning?

Machine learning refers to systems that learn patterns from data rather than relying on explicitly programmed rules. In supervised learning, a model is

trained on labeled examples to approximate a function that maps inputs to outputs. Once trained, the model is used to make predictions on new, unseen data [11].

From an operational perspective, an ML system consists of:

- **Training data:** historical examples used to learn patterns.
- **Features:** representations of raw data used as model inputs.
- **Model parameters:** learned numerical values that encode behavior.
- **Inference code:** logic that applies the trained model to new data.

Unlike traditional software, changing the data or retraining the model can change system behavior even when the code remains unchanged.

2.2 Training vs. Inference

ML systems operate in two distinct modes:

- **Training:** an offline process where model parameters are learned from data using optimization algorithms.
- **Inference:** an online or batch process where the trained model is used to generate predictions.

For example, a spam classifier may be trained weekly on historical email data but run inference in real time when new emails arrive. MLOps must manage both processes and ensure consistency between them [3].

2.3 Why Data Is Central to ML Behavior

Data is not merely an input to ML systems—it defines their behavior. If the data distribution changes over time, model performance may degrade even though the software has not changed. This dependency on data distributions is a key reason why ML systems require continuous monitoring and retraining [2].

2.4 What Are Large Language Models?

Large language models (LLMs) are a class of ML models trained on massive text corpora to predict the probability of the next token given previous tokens. Models such as GPT-style transformers learn general-purpose language representations that can be adapted to many tasks through prompting or fine-tuning [12].

Operationally, LLM-based systems often differ from traditional ML systems in that:

- Models are frequently pretrained by third parties and accessed via APIs.
- Application behavior depends heavily on prompts and system instructions.
- Outputs may be stochastic rather than deterministic.

2.5 Prompts as First-Class Artifacts

In LLM-based applications, prompts play a role similar to feature engineering or configuration in traditional ML systems. Small prompt changes can significantly alter system behavior. As a result, prompts must be versioned, evaluated, and monitored in production, making them first-class artifacts in LLMOps workflows [9].

2.6 Why These Concepts Matter for MLOps

Understanding these basic ML and LLM concepts clarifies why MLOps is necessary. Because behavior emerges from data, learned parameters, and prompts and production systems must track, validate, and monitor a broader set of artifacts. MLOps provides the processes and tooling required to manage these artifacts reliably throughout the system lifecycle.

3 What Is MLOps?

MLOps refers to a collection of practices and tooling designed to manage the end-to-end lifecycle of ML systems in production. It extends DevOps

principles by incorporating ML-specific artifacts such as datasets, features, trained models, and evaluation metrics [5].

For example, while a DevOps pipeline might compile code, run unit tests, and deploy a service, an MLOps pipeline must also validate incoming data, track experiments, register trained models, and continuously monitor prediction behavior. Google characterizes MLOps as enabling continuous delivery and automation pipelines for ML, spanning data ingestion through deployment and monitoring [6].

For example, in Google’s large-scale recommendation and ranking systems (e.g., ads, search ranking, content recommendation), models are retrained on a recurring schedule using continuously ingested user interaction data. Data validation checks ensure schema consistency and basic statistical properties before training (such as maybe user id is missing due to corrupted data, a particular data point deviating too much from the usual data distribution etc). Trained models are automatically evaluated against offline metrics (precision, recall etc on held-out dataset for example) and only promoted if they meet predefined quality thresholds. Once deployed, the models are monitored in production for input data drift (when slowly but surely the distribution of the training data is changing), prediction distribution changes, and downstream business metrics (such as click-through rate, how often users click an item after seeing it on the website). When degradation is detected, retraining or rollback is triggered through automated pipelines rather than manual intervention.

4 Why ML Systems Fail

Failures in ML systems are rarely caused by model architecture alone. Instead, they emerge from systemic issues related to data, pipelines, monitoring, and organizational practices.

4.1 Hidden Technical Debt in ML Systems

Sculley et al. [2] introduced the concept of *hidden technical debt* in ML systems, identifying several recurring patterns.

Glue code refers to ad-hoc scripts that connect data sources, feature extraction, training, and deployment. For example, a data scientist may write a quick Python script to preprocess data and later reuse it in production

without proper testing or documentation. Over time, such scripts become critical dependencies that are difficult to modify safely.

Entanglement occurs when model behavior depends on complex interactions among features, models, and downstream consumers. For instance, modifying a single feature may affect multiple models trained on shared feature pipelines, making isolated changes risky.

Undeclared consumers occur when model outputs are reused by downstream systems without explicit documentation, such as a fraud score being repurposed for credit risk assessment, amplifying the impact of model changes.

4.2 Data Quality and Training–Serving Skew

Data-related issues are among the most common causes of ML system failure. Training–serving skew occurs when the data or feature computations used during model training differ from those available at inference time [3]. This commonly arises when models are trained on complete historical batch data but deployed in real-time settings where certain features are delayed, missing, or computed differently. For example, a fraud detection model may rely during training on aggregate transaction statistics computed over fully populated databases, while in production these statistics are unavailable or only partially updated at the time of prediction, leading to systematic performance degradation despite unchanged model code.

Data leakage is a common failure mode in machine learning systems, occurring when a model is trained using information that would not be available at prediction time. For example, a loan default prediction model may inadvertently include features derived from post-loan outcomes, such as account status updates recorded after the loan decision was made, or global statistics computed using the full dataset, including future records. As a result, offline evaluation metrics appear unrealistically high, while the model fails to generalize when deployed in real-world settings where such information is unavailable.

4.3 Distribution Shift and Concept Drift

Real-world environments are non-stationary. Input distributions may change due to seasonal effects, user behavior changes, or external events. More critically, the relationship between inputs and outputs may evolve, a phenomenon

known as concept drift [4]. For example, a spam detection model trained on last year’s data may become ineffective as adversaries adapt their tactics.

Without explicit drift detection and retraining strategies, such degradation often remains unnoticed until business metrics suffer.

4.4 Monitoring Gaps and Silent Failures

Unlike traditional software systems, ML systems often fail silently. A deployed model may continue to return predictions while its accuracy, fairness, or calibration deteriorates [1]. Effective monitoring therefore requires tracking not only system uptime but also data statistics, prediction distributions, delayed labels, and downstream business outcomes.

5 The MLOps Lifecycle

The MLOps lifecycle is inherently iterative rather than linear. A typical lifecycle includes:

1. Problem formulation and definition of success metrics
2. Data collection, cleaning, and validation
3. Model development and experimentation
4. Automated training and evaluation pipelines
5. Deployment (batch or online) (Deployment refers to the process of making trained models available for inference in production, either through batch deployment, where predictions are generated periodically over large datasets, or online deployment, where predictions are served in real time in response to individual requests.)
6. Monitoring, feedback collection, and retraining

For example, a recommendation system may be retrained weekly as new interaction data arrives, with continuous monitoring to detect shifts in user behavior. This cyclic view aligns with industry MLOps frameworks [6] and extends earlier data mining process models such as CRISP-DM [8] (where Data Mining was broken into six phases: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, Deployment.)

6 MLOps and the Software Development Lifecycle

ML development increasingly occurs within standard software engineering workflows. However, ML introduces new artifacts—datasets, features, models, and evaluation results—that must be versioned and managed alongside code [1]. Unlike traditional software, requirements may evolve as data is explored, and system behavior cannot be fully specified in advance.

As a result, MLOps augments the software lifecycle by embedding experimentation, continuous training, and post-deployment monitoring into routine engineering practice.

7 Roles in MLOps and Their Place in the ML Lifecycle

Operationalizing machine learning systems requires collaboration across multiple specialized roles. Unlike traditional software projects, where responsibilities are often concentrated within a single engineering team, ML systems span data pipelines, model development, and production infrastructure. This section outlines three key roles commonly involved in MLOps: data engineer, ML engineer, and platform engineer and then explains how they contribute across the ML lifecycle.

7.1 Data Engineer

Data engineers are responsible for building and maintaining the data infrastructure that supports ML systems. Their primary focus is on ensuring that data is reliably collected, stored, processed, and made available for downstream use.

Typical responsibilities include:

- Designing data ingestion pipelines from raw sources (logs, sensors, databases)
- Cleaning, transforming, and validating data
- Managing data schemas and ensuring consistency over time

- Supporting batch and streaming data workflows

For example, in a recommendation system, data engineers may build pipelines that continuously collect user interaction events (views, clicks, purchases) and aggregate them into training datasets. Failures at this stage—such as missing fields or delayed data—directly impact model quality and reliability.

7.2 ML Engineer

ML engineers focus on turning models into production-ready systems. They bridge the gap between experimental model development and reliable deployment by integrating models with software systems and MLOps pipelines.

Typical responsibilities include:

- Implementing training and evaluation pipelines
- Ensuring training-serving consistency
- Tracking experiments and managing model versions
- Deploying models for batch or online inference
- Monitoring model performance and triggering retraining

For instance, an ML engineer may take a fraud detection model developed in a notebook and integrate it into an automated pipeline that re-trains weekly, evaluates performance against offline metrics, registers approved models, and deploys them behind an API.

7.3 Platform Engineer

Platform engineers design and maintain the underlying infrastructure that enables MLOps at scale. Their goal is to provide reusable, reliable platforms that allow data scientists and ML engineers to focus on modeling rather than infrastructure.

Typical responsibilities include:

- Building shared ML platforms and tooling
- Managing compute resources (cloud, clusters, GPUs)

- Supporting CI/CD, containerization, and orchestration
- Implementing monitoring, logging, and access control

As an example, a platform engineer may build a Kubernetes-based platform that supports automated training jobs, model serving services, and monitoring dashboards used by multiple ML teams across an organization.

7.4 MLOps Across the ML Lifecycle

MLOps connects these roles across the full ML lifecycle rather than confining responsibility to a single phase. Data engineers primarily support data collection and preparation, ML engineers focus on model development, deployment, and monitoring, while platform engineers provide the infrastructure that enables automation and scalability.

Rather than a linear handoff, MLOps encourages continuous collaboration. For example, monitoring signals detected in production may require data engineers to adjust data pipelines, ML engineers to retrain or modify models, and platform engineers to update infrastructure or tooling. This lifecycle-oriented collaboration is a defining characteristic of mature MLOps practices.

8 MLOps vs. DevOps

MLOps builds upon DevOps foundations such as automation, CI/CD pipelines, containerization, and observability. However, several distinctions necessitate specialized processes:

- Data becomes a first-class production dependency.
- Model validation relies on statistical and task-specific metrics.
- Retraining and redeployment are expected operational activities.
- Lineage and governance are required to trace models back to data and code.

For instance, rolling back a model deployment may require restoring not only code but also a specific dataset and feature pipeline.

9 From MLOps to LLMOps

Large language models introduce new operational challenges. Many LLM-based applications rely on pretrained models accessed via APIs rather than locally trained models. Consequently, LLMOps emphasizes prompt engineering, retrieval pipelines, and orchestration logic [9].

For example, modifying a prompt template can significantly change system behavior, requiring versioning and evaluation similar to model updates. Evaluation often involves human judgment, as automated metrics may not fully capture response quality or safety.

10 Governance and Risk Considerations

ML systems can introduce risks that evolve over time, including bias amplification, misuse, and unexpected feedback loops. Risk management frameworks such as the NIST AI Risk Management Framework emphasize lifecycle-based controls that align naturally with MLOps practices [10]. Embedding governance into MLOps workflows enables traceability, accountability, and compliance.

11 Conclusion

MLOps addresses the gap between experimental ML development and reliable production systems. By explicitly managing data dependencies, mitigating hidden technical debt, and enabling continuous monitoring and retraining, MLOps provides the engineering discipline necessary for modern ML systems. As AI systems increasingly rely on large foundation models, MLOps principles remain central, while LLMOps extends them to new operational contexts.

References

- [1] S. Amershi et al. Software engineering for machine learning: A case study. In *ICSE-SEIP*, 2019.
- [2] D. Sculley et al. Hidden technical debt in machine learning systems. In *NeurIPS*, 2015.

- [3] E. Breck et al. The ML test score: A rubric for ML production readiness. Google Research, 2017.
- [4] J. Gama et al. A survey on concept drift adaptation. *ACM Computing Surveys*, 2014.
- [5] M. Treveil et al. *Introducing MLOps*. O’Reilly Media, 2020.
- [6] Google Cloud. MLOps: Continuous delivery and automation pipelines in machine learning. <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>.
- [7] Microsoft Azure. What is MLOps? <https://azure.microsoft.com/en-us/solutions/machine-learning-ops>.
- [8] C. Shearer. The CRISP-DM model: The new blueprint for data mining. *Journal of Data Warehousing*, 2000.
- [9] C. Huyen. Building LLM applications for production. <https://huyenchip.com/2023/04/11/llm-engineering.html>.
- [10] NIST. AI Risk Management Framework. <https://www.nist.gov/itl/ai-risk-management-framework>.
- [11] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [12] T. B. Brown et al. Language models are few-shot learners. In *NeurIPS*, 2020.